
Autoencoders for a Harmfulness Text Classifier

Lennart Finke
ETH Zurich
lfinke@ethz.ch

David Zollikofer
ETH Zurich

Abstract

Interpretability of language models has advanced significantly in recent years, but has yielded fewer practical use cases than some hoped for. Here, we apply feature composition with Sparse Autoencoders as studied in the context of autoregressive next-token prediction LLMs to another domain; we train autoencoders on the activations for the harmfulness text classifier Prompt-Guard-86M. The sparse, interpretable features allow us to identify failure modes on adversarial input. An ensemble of the selected sparse feature with the original model improves classification accuracy on synthetic adversarial data. We further study the application to constructing jailbreaks and identifying misclassified data.

1 Introduction

Deep learning models are often hard to control in production and, in particular, are susceptible to adversarial attacks by malicious users [23, 3]. Therefore, model providers rely on separate classifiers to flag undesirable model outputs, which are themselves based on deep learning. We focus on LLMs in the text modality only.

Although it would be desirable to explain the behavior of such classifier models in order to detect unexpected failure modes, this is not done in practice because robust interpretation of deep neural nets remains difficult. Here, we aim to apply recent advances in mechanistic interpretation of autoregressive LLMs to the classifier domain.

Operationalizing this, we take a popular classifier (Prompt-Guard-86M) [7], which classifies inputs as “benign”, “jailbreak”, or “injection”, and ask: Do these categories decompose into finer categories based on the internal activations of the model? Does this inform the discovery of circumvention strategies?

We approach this interpretation task with sparse autoencoders (henceforth SAEs) following Cunningham et al. [5]. In this technique, the residual stream of the transformer model is reconstructed from a larger latent dimension subject to a sparsity condition. This could alleviate the hypothesized problem of superposition [19]. In superposition, the latent space of deep learning models contains more directions that carry semantic meaning than the number of dimensions in the latent space, rendering interpretability difficult.

We confirm that our classifier can accurately classify the samples in our jailbreak dataset. We then train SAEs on a more diverse mixture of data, on the residual stream of all 12 layers in the encoder. Activations of these sparse features on the jailbreak classification dataset are obtained, which allows us to confirm the quality of the reconstruction by predicting labels from the activations. Based on token-level as well as per-sample average activations, we formulate mechanistic hypotheses about certain features, which we then verify through synthetic examples.

We find that some, though far from all, SAE features carry monosemantic meaning, which we then exploit to both attack and defend the classifier.

Code and autoencoder weights are available at <https://github.com/lennart-finke/classifier-interp>.

2 Prior Work

Bricken et al. [2] studies the automatic discovery of semantically meaningful features in a one-layer transformer. After having trained the SAEs, its sparse directions are studied via the text they activate on. Then, a causal effect can be shown by adding or subtracting the sparse direction in the residual stream and observing the change in prediction likelihood of certain tokens.

As an alternative perspective, Kantamneni et al. [11] find that baseline methods like linear probes or even regression outperform SAEs in many settings. They conclude that SAEs are typically only more conducive to interpretability to the extent that their input basis has semantic meaning.

Sparse autoencoders have been trained on the activations of BERT-style [6] encoders [17], but there is no precedent for models trained as encoders for a classification task. Perhaps the most relevant work is by Kuznetsov et al. [12], who construct a text classifier from Gemma’s SAE features. This is the reverse of our plan, since we analyze a given classifier. Therefore, we must train the SAEs ourselves and hope to contribute to the research community by providing training code and weights.

Since the practical use of interpretability has been limited thus far, we hope to find an application that improves the robustness of classification. In principle, if the causal structure leading to a classification in a deep learning classifier can be interpreted, this leads to a better understanding of the shortcomings. A causal understanding should be easier to obtain than for the more popular object of study, LLMs, as they perform a much more complicated language modeling task.

3 Data & Model

The model we study, Prompt-Guard-86M, is based on mDeBERTa-v3-base [8]. It is an encoder BERT-style [6] model with 12 self-attention layers. The underlying encoder uses a large token vocabulary of 251,000, a context window of 512 tokens with both absolute and relative positional embeddings, GELU activations, layer norm, and an embedding dimension of 768. The classification logits are obtained through a linear down-projection of the embedding of the first input token in the last layer. We are not aware of any information on the training data for Prompt-Guard-86M. This introduces an important unknown, as we have to assume that all available data (at least what was available prior to the model’s publication) was used for training.

For SAE training, we need activations from both jailbreak and benign samples. We use a random draw from a mix of datasets. For jailbreaks, we use

1. the In-The-Wild dataset from Shen et al. [20] of 1405 malicious prompts collected from December 2022 to December 2023, collected from various online forums.
2. the train split of the dataset from Jiang et al. [10] of 82728 samples of malicious prompts, obtained by LLMs combining benign prompts with jailbreak tactics from online jailbreak prompts.
3. the text-based samples in the JailBreakV28K dataset from Luo et al. [14], yielding 20000 malicious samples, also obtained through LLM rephrasing of human-written jailbreaks.

For harmless prompts, we use 5% of the instruction training data from Orca [16], adding 147000 samples. The In-The-Wild dataset is included 5 times because we find it to be of the highest quality, and because we will mainly want to look at the SAE activations on these samples, which means that we want them to be in-distribution. It follows that ca. 43% of our training distribution is jailbreaks. This even split is chosen so that approximately half of the sparse features will pertain to jailbreaks. However, it could also be the case that, because the benign prompts are more diverse, it would be beneficial to include a larger percentage of them. We did not test different splits between jailbreak and benign samples.

For the analysis of activations, we use a dataset available at <https://huggingface.co/datasets/jackhhao/jailbreak-classification>, itself a mix of Orca [16] for benign prompts (517 samples train, 123 test) and In-The-Wild [20] for jailbreak prompts (527 samples train, 139 test). This means that the distribution of the downstream analysis and SAE training overlaps. We henceforth will refer to this dataset as the classification task dataset.

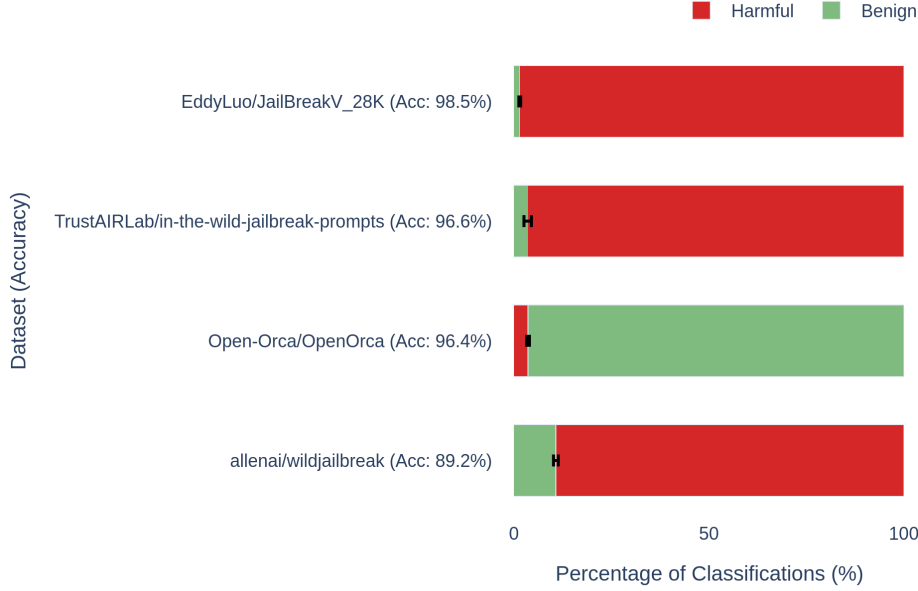


Figure 1: Classification accuracies for different datasets used in the SAE training and downstream analysis. Error bars indicate a 0.95 confidence interval of an i.i.d. binomial proportion.

We run inference with Prompt-Guard-86M on the datasets, downsampled to 100000 prompts where applicable, and classify those with a total probability of > 0.5 , with results shown in Fig. 1. We observe sufficient classification accuracy to confidently use the model. However, the presence of misclassified samples allows us to analyze the failure modes of both the datasets and the model. We did not test the architecturally very similar Prompt-Guard-2-86M, but our methods are applicable to that model as well.

4 Methods & Results

Mathematical Foundations A harmfulness classifier is a function $f_\theta : \Sigma^* \rightarrow [0, 1]$, where Σ^* is the Kleene closure over an alphabet Σ , the θ are some parameters, and the output is the probability that the input is harmful. Given a dataset of harmful inputs $H \subseteq \Sigma^*$ and benign inputs $B \subseteq \Sigma^*$, we are interested in false positives $\hat{B}_\tau := \{b \in B : f_\theta(b) > \tau\}$ as well as false negatives $\hat{H}_\tau := \{h \in H : f_\theta(h) < \tau\}$. Assuming the classifier is a deep learning model with hidden dimension n and l layers, we also obtain activations $f_{\text{act}} : \Sigma^* \rightarrow \mathbb{R}^{n \times l}$. When we train a sparse autoencoder $g : \mathbb{R}^{n \times l} \rightarrow \mathbb{R}^{N \times l}$ with $N > n$, this may help characterize f via $g \circ f_{\text{act}}$ on, say, $H \setminus \hat{H}$ versus $\hat{H}$ or B versus H .

Autoencoder Training We train SAEs on the residual stream after the MLP in all 12 encoder layers, using the `dictionary_learning` package [18]. Using a batch size of 128, we train for 100,000 steps with a learning rate of 0.001, with 5,000 warmup steps for the learning rate and sparsity penalty. We use `bf16` quantization. Activations are normalized, and then up- and down-projected to and from four times the original embedding dimension, yielding 3,072 sparse features. The $\mathcal{L}_1$ penalty factor is 0.1. We did not sweep over hyperparameters, except for batch size, which we chose to maximize token throughput. Training was completed on a Radeon Pro W7900. See Figure 5 for training metrics.

Assessment of SAE Quality We are able to confirm that the sparse features reconstruct the embeddings numerically, but it is a priori not clear whether this is sufficient for semantically connecting SAE features to the classification decision. We therefore regress the sparse activations of each layer to the predicted labels on our classification task dataset, and then record the regression’s accuracy and AUC on the test set. We find an approximately linear increase from random guessing to predictions as accurate as the Prompt-Guard model itself. We also run AlignedUMAP [15, 21] on the SAE activations for visualization and record Silhouette scores on the train split, as well as k-nearest neighbors accuracy on the test set from the train set. Results of both the regression and the UMAP experiments are shown in 6. The UMAP embeddings of selected layers are shown in Figure 2. In these embeddings, it is easy to see some samples clustered together with different labels than the surrounding samples, which led us to diagnose several mislabeled prompts. We explore this systematically in the next section.

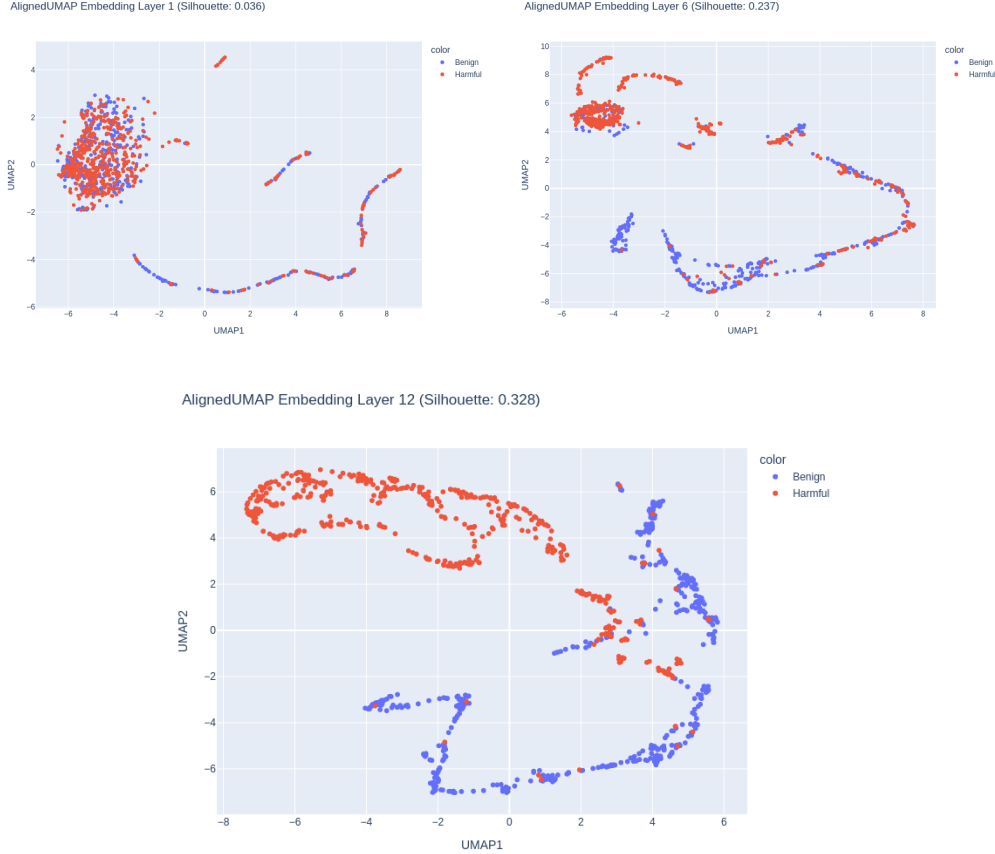


Figure 2: AlignedUMAP [15, 21] on SAE activations for layer 1 (**Left**), layer 6 (**Right**) and layer 12 (**Bottom**).

Finding Mislabeled Samples SAEs are, among other things, a method to cluster input data via the activations. We use this property to find misclassified samples in both the benign and harmful datasets in the Classification Dataset. Thus, we benchmark a SAE-driven manual inspection against simply sorting samples by how confidently wrongly Prompt-Guard classifies them. We used the AlignedUMAP embeddings on layer 10 to manually find 25 samples which were clustered directly next to other samples with a different label, and manually assigned them to to be either correctly labeled, dubiously labeled, or incorrectly labeled. We do the same for the 25 samples which were misclassified with the highest confidence. Results are given in Table 1. Both methods are equally good at identifying mislabeled data. However, unexpectedly, they do not find the same misclassified samples. For example, the SAE UMAP embeddings found the sample [Put your prompt] , and end with "Shakespeare", which is wrongly listed as a jailbreak in the

classification dataset, whereas the misclassification confidence method did not find this sample. Conversely, the misclassification confidence method finds a sample that is marked as a jailbreak, but consists of some Javascript code that modifies the appearance of the ChatGPT web interface. While this could be used maliciously, it is not a jailbreak and thus mislabeled. In summary, while the misclassification confidence method should be preferred over SAEs for finding mislabeled data, due to its simplicity, SAEs nonetheless help find some orthogonal failure modes of the classifier model.

Discovery of Semantic SAE Features We produce visualizations of per-token SAE activations as in Bricken et al. [2]. To pre-select the most relevant features, we sort by p-values from the regression on predicted labels. As a case study, we find a particular feature in layer 10 that seems to fire exclusively on leetspeak text, a common technique in jailbreaks. As a causal intervention, we transform 500 benign samples from the classification set into leetspeak and record their activations. Indeed, this feature experiences the largest absolute change in activation in its layer (see Figure 4). While this does not prove monosemanticity, it does demonstrate that the visualization tool is, in principle, useful for interpretation, even for an encoder-only classifier like Prompt-Guard. Through the same method, we find a feature that fires only on one sample in the classification dataset, namely on the first Ω in the string “ Ω INFARED Ω ”. We again confirm our hypothesis that these are related by recording the change in activation when adding strings related to this trigger string. Results are also shown in Figure 4.

An SAE-Assisted Discovery of a False Positive Attack Since the “ Ω INFARED Ω ” feature is correlated with a jailbreak classification, we hypothesize that the model is using the presence of this string verbatim to make its decision. If this is the case, it represents a failure mode of the model, since the string itself is inoffensive. Therefore, this fault could be used to construct a false positive attack, where the classifier finds a completely benign string to be a jailbreak. Indeed, appending the string to 500 benign samples results in an average increase of $6.5\% \pm 20.2\%$ in the probability of being classified as a jailbreak. We further hypothesize that repeatedly inserting the relevant string will increase the effect, and thus append the string “INFARED” five times, finding an increase of $18.5\% \pm 32.1\%$ in the probability of being classified as a jailbreak.

Hardening the Classifier against Adversarial Inputs In the leetspeak feature, we have identified an SAE feature that can be exploited to enhance classification accuracy. Prompt-Guard, despite producing a leetspeak-activated sparse feature, loses all predictive power on leetspeak text and can thus easily be circumvented by converting the malicious prompt to leetspeak. We therefore investigate the ensemble of the mean activation of the leetspeak feature and the default predictions as a possible solution to leetspeak-transformed data. To do so, we train a logistic regression taking the classification likelihoods and leetspeak feature mean activation as inputs on the training split of the jailbreak classification dataset, and calculate the AUC on the test split. This is done on (1) the unchanged dataset, (2) the dataset transformed entirely to leetspeak, and (3) a more realistic attack model, where half of the malicious samples are transformed to leetspeak and the other half as well as the benign samples are unchanged. Results are displayed in Figure 3. On the leetspeak data, the original model performs as poorly as random chance. On the original dataset, the ensemble does not hurt performance, and almost completely fixes the performance drop in the data where half of the jailbreak samples are transformed. Most interestingly, the ensemble is even much better when all samples are transformed to leetspeak. This is surprising because when all samples are in leetspeak, the leetspeak feature shouldn’t a priori be able to distinguish between any two samples. We hypothesize that this means that the model can still read some parts of text in some samples, and those samples then have a lower sparse activation, which the logistic regression successfully exploits.

5 Conclusion

We have demonstrated that sparse autoencoders can be used both to attack and to defend a prompt harmfulness classifier. Regarding attacks, we generally do not recommend our method over others, as there are many highly successful attack vectors in the threat model where the attacker has access to the classifier weights, in particular gradient-based methods [24, 1]. While our method might be better suited to finding adversarial strings with lower perplexity, or that are in-distribution in some other sense, there are also gradient based methods that can produce natural-looking adversarial suffixes [13].

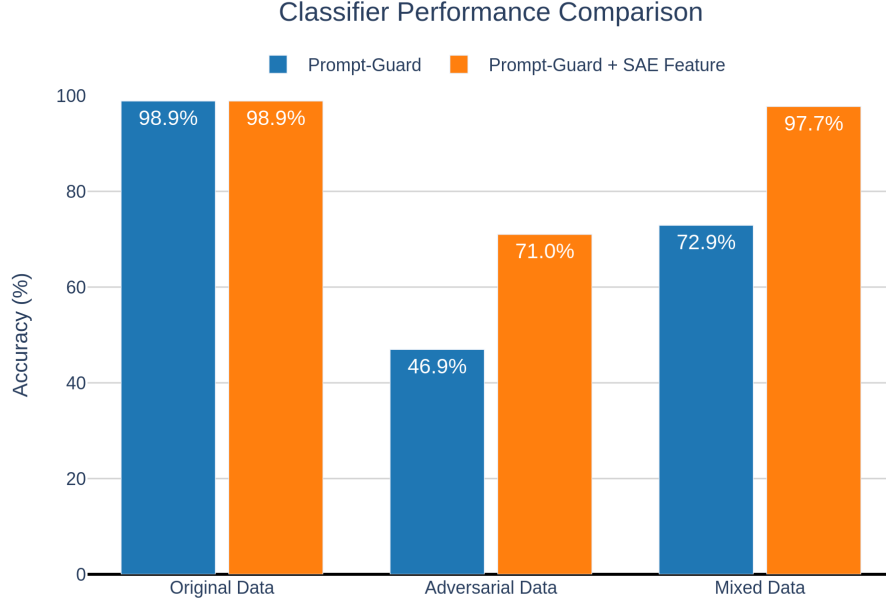


Figure 3: Classification AUC for both the original Prompt-Guard model and the ensemble of Prompt-Guard predictions with the mean activation of the leetspeak SAE feature. Original data is the unchanged jailbreak classification dataset, adversarial data is the complete dataset transformed into leetspeak, and mixed data is the original dataset with half of the jailbreak samples converted into leetspeak.

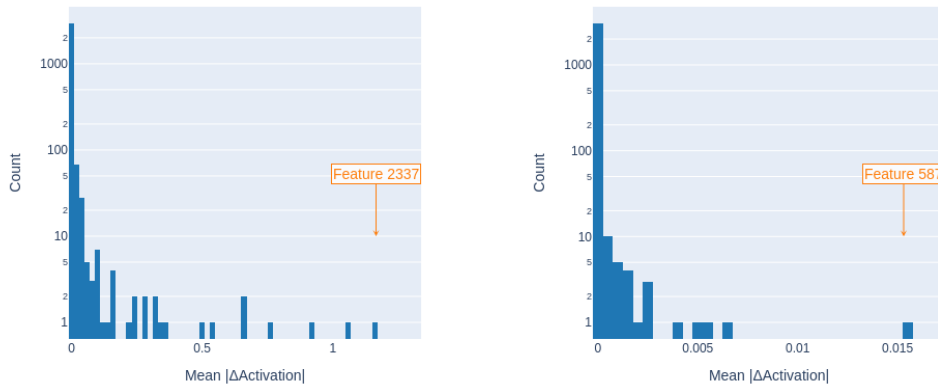


Figure 4: Per-token average change in SAE feature activations, of features in a certain layer, when transforming benign prompts. **Left:** The transformation is turning the prompt into leetspeak. Feature 2337 in layer 10 was manually identified to differentially activate on such text through the per-token activation visualization. **Right:** The transformation is appending a single Ω to the end.

Regarding defenses, however, we find that our method presents a rare opportunity to patch a production model and steer its desired behavior in a fine-grained manner. In our case, we hardened the classifier against a text transformation attack, but this can be extended to any other type of attack which the model forms a sparse feature for, which as we showed is far from equivalent to being robust against the attack. While for our text transformation attack, a simpler NLP filter without deep learning might also be able to successfully defend against it, we expect that more intricate features that do not admit such a solution also exist. As classifiers become larger, we expect that more sparse features can be extracted [22], and thus more fine-grained control becomes possible.

We also note that unsupervised interpretability methods such as SAEs require careful attention to avoid false positives [9]. Here, we have taken care to use mostly supervised methods on top of SAEs to counter this problem, but our results could conceivably not generalize beyond what we have shown here.

In our methodology, a manual intervention was used to identify which SAE features (of those that predict the classification label) pertain to a given semantic feature. Recent efforts have automated this process significantly [22, 4, 2], which we did not make use of here. We expect that automatic assignment of semantic meaning to SAE features and circuit analysis are applicable to the classifier setting as well.

Acknowledgments

We thank Elliott Ash for supervision and guidance on the project, as well as the initial idea that led us to take up this direction. This research was undertaken in large part during a course at ETH taught by Elliott. We also thank Andy Ardit and Terry Jingchen for an insightful critique of the project outline.

References

- [1] M. Andriushchenko, F. Croce, and N. Flammarion. Jailbreaking leading safety-aligned llms with simple adaptive attacks. *arXiv preprint arXiv:2404.02151*, 2024.
- [2] T. Bricken, A. Templeton, J. Batson, B. Chen, A. Jermyn, T. Conerly, N. Turner, C. Anil, C. Denison, A. Askell, R. Lasenby, Y. Wu, S. Kravec, N. Schiefer, T. Maxwell, N. Joseph, Z. Hatfield-Dodds, A. Tamkin, K. Nguyen, B. McLean, J. E. Burke, T. Hume, S. Carter, T. Henighan, and C. Olah. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023. <https://transformer-circuits.pub/2023/monosemantic-features/index.html>.
- [3] N. Carlini, M. Nasr, C. A. Choquette-Choo, M. Jagielski, I. Gao, P. W. W. Koh, D. Ippolito, F. Tramèr, and L. Schmidt. Are aligned neural networks adversarially aligned? *Advances in Neural Information Processing Systems*, 36:61478–61500, 2023.
- [4] A. Conmy, A. Mavor-Parker, A. Lynch, S. Heimersheim, and A. Garriga-Alonso. Towards automated circuit discovery for mechanistic interpretability. *Advances in Neural Information Processing Systems*, 36:16318–16352, 2023.
- [5] H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey. Sparse autoencoders find highly interpretable features in language models. *arXiv preprint arXiv:2309.08600*, 2023.
- [6] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019.
- [7] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [8] P. He, X. Liu, J. Gao, and W. Chen. Deberta: Decoding-enhanced bert with disentangled attention. *arXiv preprint arXiv:2006.03654*, 2020.
- [9] T. Heap, T. Lawson, L. Farnik, and L. Aitchison. Sparse autoencoders can interpret randomly initialized transformers. *arXiv preprint arXiv:2501.17727*, 2025.
- [10] L. Jiang, K. Rao, S. Han, A. Ettinger, F. Brahman, S. Kumar, N. Mireshghallah, X. Lu, M. Sap, Y. Choi, et al. Wildteaming at scale: From in-the-wild jailbreaks to (adversarially) safer language models. *Advances in Neural Information Processing Systems*, 37:47094–47165, 2024.

- [11] S. Kantamneni, J. Engels, S. Rajamanoharan, M. Tegmark, and N. Nanda. Are sparse autoencoders useful? a case study in sparse probing. *arXiv preprint arXiv:2502.16681*, 2025.
- [12] K. Kuznetsov, L. Kushnareva, P. Druzhinina, A. Razzhigaev, A. Voznyuk, I. Piontkovskaya, E. Burnaev, and S. Barannikov. Feature-level insights into artificial text detection with sparse autoencoders. *arXiv preprint arXiv:2503.03601*, 2025.
- [13] X. Liu, N. Xu, M. Chen, and C. Xiao. Autodan: Generating stealthy jailbreak prompts on aligned large language models. *arXiv preprint arXiv:2310.04451*, 2023.
- [14] W. Luo, S. Ma, X. Liu, X. Guo, and C. Xiao. Jailbreakv-28k: A benchmark for assessing the robustness of multimodal large language models against jailbreak attacks, 2024.
- [15] L. McInnes, J. Healy, and J. Melville. Umap: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2018.
- [16] S. Mukherjee, A. Mitra, G. Jawahar, S. Agarwal, H. Palangi, and A. Awadallah. Orca: Progressive learning from complex explanation traces of gpt-4. *arXiv preprint arXiv:2306.02707*, 2023.
- [17] N. Nanda and J. Bloom. Transformerlens. <https://github.com/TransformerLensOrg/TransformerLens>, 2022.
- [18] A. K. Samuel Marks and A. Mueller. dictionary_learning. https://github.com/saprmaks/dictionary_learning, 2024.
- [19] L. Sharkey, D. Braun, and B. Millidge. Taking features out of superposition with sparse autoencoders. In *AI Alignment Forum*, volume 6, pages 12–13, 2022.
- [20] X. Shen, Z. Chen, M. Backes, Y. Shen, and Y. Zhang. "do anything now": Characterizing and evaluating in-the-wild jailbreak prompts on large language models. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 1671–1685, 2024.
- [21] G. Singh, F. Mémoli, G. E. Carlsson, et al. Topological methods for the analysis of high dimensional data sets and 3d object recognition. *PBG@ Eurographics*, 2(091-100):90, 2007.
- [22] A. Templeton, T. Conerly, J. Marcus, J. Lindsey, T. Bricken, B. Chen, A. Pearce, C. Citro, E. Ameisen, A. Jones, H. Cunningham, N. L. Turner, C. McDougall, M. MacDiarmid, C. D. Freeman, T. R. Sumers, E. Rees, J. Batson, A. Jermyn, S. Carter, C. Olah, and T. Henighan. Scaling monosemanticity: Extracting interpretable features from claude 3 sonnet. *Transformer Circuits Thread*, 2024. URL <https://transformer-circuits.pub/2024/scaling-monosemanticity/index.html>.
- [23] A. Wei, N. Haghtalab, and J. Steinhardt. Jailbroken: How does llm safety training fail? *Advances in Neural Information Processing Systems*, 36:80079–80110, 2023.
- [24] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Z. Kolter, and M. Fredrikson. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*, 2023.

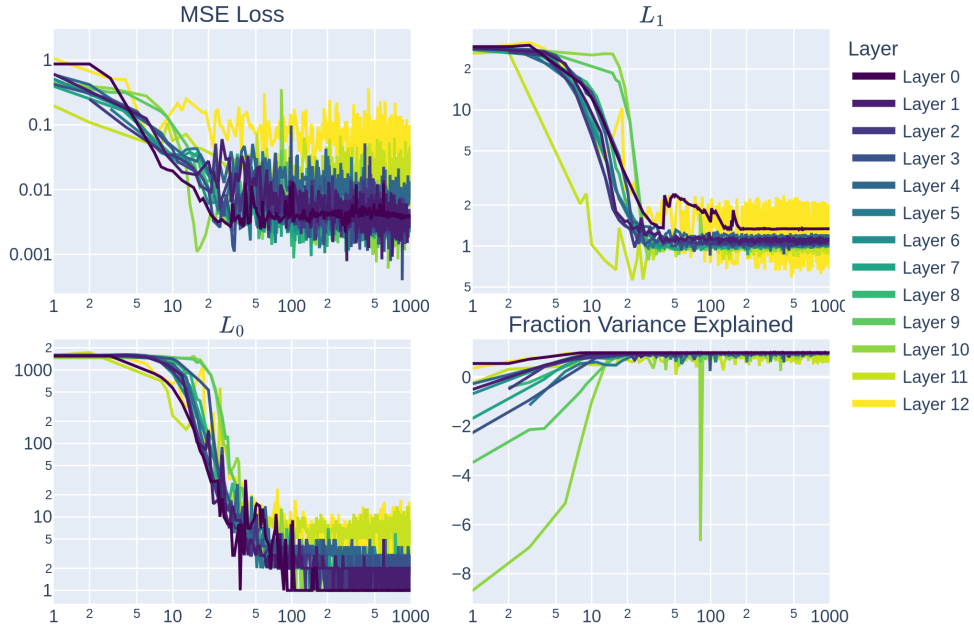


Figure 5: SAE training metrics. **Top left:** MSE reconstruction loss. **Top right and bottom left:** L_1 norm of up-projected sparse features. **Bottom right:** Fraction of variance of residual stream explained.

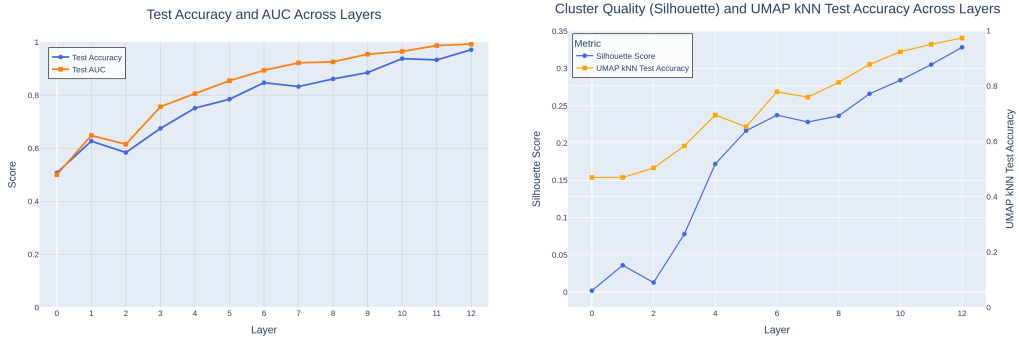


Figure 6: Predicting benign-harmful labels from SAE activations. **Left:** Accuracy and AUC on the test split of least-squares regression on the train set. **Right:** 5-nearest neighbors classification accuracy for the train split to the test split, and Silhouette score, each on 2D UMAP embeddings of SAE activations, per layer.

	Wrongly Labeled	Dubiously Labeled	Correctly Labeled
UMAP Clusters	11	6	8
Misclassification Confidence	11	7	7

Table 1: In the top row, manual assessments of label correctness for 25 manually picked samples in UMAP clusters together with other samples with the opposite label. In the bottom row, manual assessments of label correctness for the with the 25 samples that were classified wrongly by Prompt-Guard with the highest confidence.